Data-Centric Analysis of Hallucination and Confidence Calibration in Large Language Models

Dan Shamia
Tel Aviv University
danshamia@mail.tau.ac.il

Daniel Halperin
Tel Aviv University
danielh3@mail.tau.ac.il

Abstract

Large Language Models (LLMs) are increasingly deployed in high-stakes environments, yet their tendency to hallucinate—generating confident but factually incorrect outputs—remains a critical reliability concern. In this work, we investigate how the frequency of factual information in training data affects both accuracy and confidence calibration. Using the FEVER dataset, we construct frequency-stratified splits separating high-exposure from low-exposure facts based on Wikipedia page reference frequency. We fine-tune Meta-Llama-3.1-8B-Instruct using QLoRA on a balanced training set of 18,082 examples equally distributed across frequency buckets, and evaluate on three distinct test sets. Our results reveal a fundamental accuracy–calibration trade-off: fine-tuning improves overall accuracy by 6.8 percentage points (81.9% $\rightarrow$ 88.7%) and nearly eliminates the frequency-based accuracy gap (from 2.1pp to 0.3pp), but worsens calibration—Expected Calibration Error roughly triples and the model becomes systematically overconfident by $\sim$3pp. Notably, despite this overconfidence, the rate of high-confidence errors actually decreases*, particularly on low-frequency facts ($-$2.3pp). These findings demonstrate that data frequency distribution directly impacts both hallucination rates and calibration, with important implications for LLM deployment.*

1. Introduction

The remarkable capabilities of Large Language Models (LLMs) have led to their rapid adoption across domains from healthcare to legal analysis. However, a persistent failure mode threatens their reliability: *hallucination*—the generation of plausible-sounding but factually incorrect content [9]. When coupled with high confidence scores, hallucinations become particularly dangerous, as downstream users and automated systems may trust incorrect outputs without question.

The problem is compounded by the scale at which LLMs operate. A model that halluccinates 5% of the time but serves millions of queries daily produces tens of thousands of false statements. If those false statements are accompanied by high confidence—what we term *high-confidence errors*—the potential for harm multiplies, as users are less likely to seek verification.

Recent work has shown that fine-tuning on new knowledge can paradoxically *encourage* hallucinations [2], while theoretical analysis argues hallucination is an inherent limitation of statistical modeling [8]. Separately, Guo et al. [3] established that modern neural networks are systematically miscalibrated, and Kandpal et al. [5] demonstrated that LLM accuracy correlates strongly with entity frequency in pre-training data.

Despite these individual findings, the *intersection* of data frequency, hallucination, and calibration remains underexplored. Do models hallucinate more on rare facts? When they do, are they appropriately uncertain? Can targeted fine-tuning fix these issues, or does it introduce new calibration problems?

In this work, we bridge these research areas by asking: **How does the frequency of factual training data affect both accuracy and confidence calibration?** We formulate three concrete hypotheses:

H1: Models achieve higher accuracy on high-frequency (commonly encountered) facts than on low-frequency (long-tail) facts.

H2: Fine-tuning improves accuracy but degrades confidence calibration, consistent with Guo *et al*. [3].

H3: Low-frequency facts are more susceptible to high-confidence errors—the model is more likely to be confidently wrong on rare facts.

To test these hypotheses, we design a controlled experiment using the FEVER dataset [7] with frequency-stratified splits, fine-tune Llama-3.1-8B-Instruct using QLoRA [1], and evaluate across a 2×3 matrix (baseline *vs*. fine-tuned $\times$ high/low/mixed frequency test sets). Our evaluation goes beyond standard accuracy to include Expected Calibration Error (ECE), high-confidence error rates (HCE), overconfi-

dence gap analysis, and reliability diagrams.

Our key contributions are:

- A controlled, data-centric experimental framework that isolates the effect of fact frequency on LLM behavior, using percentile-based bucketing on Wikipedia page references.
- Empirical evidence of a fundamental accuracy–calibration trade-off: fine-tuning improves accuracy by 6.8pp but roughly triples ECE.
- A novel distinction between *global miscalibration* (ECE) and *harmful miscalibration* (HCE), showing that fine-tuning worsens the former but improves the latter.
- Evidence that frequency-balanced training nearly eliminates the accuracy gap between high- and low-frequency facts (2.1pp $\rightarrow$ 0.3pp).

2. Related Work

Hallucination in LLMs. Zhang et al. [9] provide a comprehensive taxonomy of hallucination in LLMs, distinguishing between *factuality hallucinations* (contradicting established facts) and *faithfulness hallucinations* (deviating from provided context). Our work focuses on factuality hallucinations in a controlled claim verification setting, where ground truth is well-defined through the FEVER dataset. By using a binary classification task (Supports/Refutes) rather than open-ended generation, we can precisely measure hallucination rates and correlate them with data frequency.

Fine-tuning and Hallucination. Gekhman et al. [2] demonstrated that fine-tuning LLMs on examples containing *new knowledge*—facts not present in pre-training data—can encourage hallucinations even on unrelated topics. This finding is central to our motivation: if the model's familiarity with facts (as proxied by frequency) affects hallucination, then deliberately manipulating the frequency distribution of training data should reveal this relationship. Our frequency-stratified approach directly tests whether exposure level modulates the fine-tuning–hallucination dynamic.

Inherent Limitations of LLMs. Xu et al. [8] argue that hallucination is an *inevitable* consequence of statistical language modeling, as LLMs learn distributional patterns rather than grounded truth. This theoretical perspective motivates our focus on calibration as a practical mitigation: if hallucinations cannot be eliminated entirely, models should at least express appropriate uncertainty when they are likely to be wrong.

Confidence Calibration in Neural Networks. Guo et al. [3] established that modern deep neural networks are systematically poorly calibrated, typically exhibiting overconfidence, and proposed Expected Calibration Error (ECE) as a standard evaluation metric along with temperature scaling as a post-hoc correction. Their work focused on image classification; we extend the calibration analysis to LLMs in a fact verification setting. We adopt ECE as our primary calibration metric and additionally introduce the *high-confidence error rate* (HCE)—the fraction of predictions that are both wrong and highly confident—to capture the most dangerous deployment failure mode.

Frequency and LLM Knowledge. Kandpal et al. [5] showed that LLM accuracy on factual questions strongly correlates with how often the relevant entity appears in pre-training data, with long-tail entities receiving substantially lower accuracy. Mallen et al. [6] corroborated this finding, demonstrating that parametric (internal) memory is unreliable for infrequent entities and that retrieval augmentation is necessary for the long tail. Our work extends this line of research by examining not just accuracy but also *calibration* across frequency strata, revealing that the accuracy–frequency relationship interacts with confidence in non-obvious ways—specifically, that fine-tuning can improve accuracy while simultaneously degrading calibration.

Our Position. Unlike prior work that studies hallucination detection, calibration, or frequency effects in isolation, we provide a unified data-centric analysis that jointly examines all three phenomena. Our frequency-stratified experimental design enables direct comparison of model behavior on high-exposure *vs*. low-exposure facts across multiple calibration metrics, going beyond aggregate statistics to reveal frequency-dependent calibration patterns.

3. Data

3.1. Dataset

We use the FEVER (Fact Extraction and VERification) dataset v1.0 [7], a large-scale benchmark containing claims derived from Wikipedia along with labels indicating whether each claim is SUPPORTED, REFUTED, or has NOT ENOUGH INFO based on Wikipedia evidence. FEVER's training split contains 145,449 examples and the labeled development split contains 19,998 examples.

We filter to binary classification, retaining only SUPPORTS and REFUTES labels and discarding all NOT ENOUGH INFO (NEI) examples. NEI claims lack supporting evidence in the dataset, making them unsuitable for a controlled factual verification task where every example must have a clear ground-truth label. This filtering yields a substantial pool of examples for constructing balanced, frequency-stratified splits.

Critically, we present claims *without* the Wikipedia evidence passages that accompany them in FEVER. The model receives only the claim text and must classify it

based entirely on its parametric (internal) knowledge. This is a deliberate design choice: if we provided evidence, we would be testing *reading comprehension*—whether the model can match a claim to a given passage (textual entailment). Any model could do this regardless of prior knowledge, because the answer would be contained in the evidence. By withholding evidence, we instead test *factual recall*—whether the model actually "knows" the fact from its pre-training. This is what makes frequency relevant: a claim about a frequently-referenced Wikipedia page (e.g., "Barack Obama") was likely encountered many times during pre-training, so the model can answer from memory. A claim about a rarely-referenced page may require the model to guess, exposing gaps in its parametric knowledge.

3.2. Frequency Bucketing

The central data design choice is *frequency-based stratification*. Our approach:

1. For every Wikipedia page referenced in the FEVER training split, count how many claims reference it, producing a page-level frequency distribution.
2. Compute the 33rd and 67th percentiles of *unique page* frequencies (not per-example frequencies, to avoid skew from highly-referenced pages like "United States" or "Barack Obama").
3. Define three buckets using these data-driven thresholds:
   - **Low**: page frequency ≤ 2 (bottom third)
   - **Medium**: $2 <$ page frequency ≤ 7 (middle third)
   - **High**: page frequency > 7 (top third)

The same train-derived thresholds are applied to the development set, ensuring consistent bucketing across splits. This percentile-based approach adapts to the actual data distribution rather than using arbitrary fixed thresholds.

An important property of our bucketing: FEVER page frequency serves as a *proxy* for real-world topic prominence. Wikipedia pages that are referenced by many FEVER claims tend to be about well-known topics (major historical events, famous individuals, popular media) that also appear frequently in the LLM's pre-training corpus. This is validated by our results showing that even the zero-shot baseline exhibits higher accuracy on high-frequency facts—despite never having seen FEVER data.

3.3. Data Splits

- **Training set** (from FEVER train): 18,082 examples, balanced across the three frequency buckets ($\sim$6,666 each) with 50/50 label balance (Supports/Refutes) within each bucket. The balanced design ensures the model receives equal exposure to high-, medium-, and low-frequency facts during fine-tuning.
- **Test-High**: 1,000 examples from the high-frequency bucket (500 Supports, 500 Refutes).
- **Test-Low**: 1,000 examples from the low-frequency bucket (500 Supports, 500 Refutes).
- **Test-Mixed**: 996 examples ($\sim$333 per bucket, label-balanced within each).

All test sets are drawn from the FEVER development split with no claim overlap with the training set. A fixed random seed (42) ensures full reproducibility.

3.4. Prompt Format

Each example is formatted as a claim-only instruction prompt:

```
### Instruction:
Based on your knowledge, classify the
claim as SUPPORTS (true) or REFUTES (false).

### Claim:
{claim text}

### Answer:
{SUPPORTS or REFUTES}
```

During training, the label is appended as the completion target. During evaluation, the prompt ends at "`### Answer:`" and the model's next-token logits are used for prediction and confidence estimation.

4. Methods

4.1. Base Model

We use Meta-Llama-3.1-8B-Instruct as our base model. This instruction-tuned variant of the Llama 3.1 family offers strong zero-shot performance across diverse NLP tasks while remaining feasible for academic-scale fine-tuning. With 8 billion parameters, it represents a practical balance between model capability and computational requirements.

4.2. QLoRA: Quantization and Low-Rank Adaptation

To enable fine-tuning on a single GPU, we employ QLoRA [1], combining two complementary techniques:

4-Bit Quantization. The base model weights are quantized to 4-bit precision using NF4 (Normalized Float 4-bit) via the `bitsandbytes` library. We enable double quantization—quantizing the quantization constants themselves—which further reduces memory with negligible quality loss. Computation is performed in bfloat16 for numerical stability.

Low-Rank Adaptation (LoRA). Rather than updating all 8 billion parameters, we attach low-rank adapter matrices to selected weight matrices [4]. For each target weight $W \in \mathbb{R}^{d\times k}$, LoRA introduces a low-rank decomposition $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank

$r \ll \min(d, k)$. During forward pass: $h = (W + \frac{\alpha}{r}BA)x$, where α is a scaling factor and only A and B are trainable.

We attach LoRA adapters to *all* attention and MLP projection layers:

- Attention: `q_proj`, `k_proj`, `v_proj`, `o_proj`
- MLP: `gate_proj`, `up_proj`, `down_proj`

This comprehensive targeting (7 modules per transformer layer) provides more expressive fine-tuning than the common practice of targeting only attention layers. Configuration: rank r=16, scaling α=32 (effective scale $\alpha/r = 2$), dropout 0.05.

4.3. Training Configuration

We use the HuggingFace `Trainer` with the following configuration:

Parameter	Value
Epochs	2
Batch size (per device)	8
Gradient accumulation	2 (effective batch = 16)
Learning rate	2×10^{-4}
LR scheduler	Cosine decay
Warmup steps	10
Max sequence length	256 tokens
Precision	bfloat16
Best model selection	Lowest eval loss

Table 1. Training hyperparameters.

The training objective is standard causal language modeling (next-token prediction), where labels are a copy of the input token IDs. The model thus learns to generate the complete prompt–answer sequence, including the final label token.

4.4. Evaluation Protocol

For each test claim, we construct the prompt (without the answer) and perform a single forward pass. We extract the logits at the final token position for the "SUPPORTS" and "REFUTES" label tokens, and compute confidence via softmax:

$$\text{conf} = \max\left(\text{softmax}\left(\begin{bmatrix} z_{\text{SUP}} \\ z_{\text{REF}} \end{bmatrix}\right)\right) \tag{1}$$

where z_{SUP} and z_{REF} are the logits for the respective label tokens. This approach provides a principled probability estimate directly from the model's output distribution, without requiring generation-based sampling.

4.5. Metrics

We evaluate four complementary metrics:

- **Accuracy**: fraction of correct predictions.

	Test-High	Test-Low	Test-Mixed
Baseline	84.4%	82.3%	78.9%
Fine-tuned	89.2%	88.9%	88.0%
Δ	+4.8pp	+6.6pp	+9.1pp

Table 2. Accuracy across experimental conditions. Fine-tuning improves all conditions and nearly eliminates the frequency gap (2.1pp $\rightarrow$ 0.3pp).

- **Expected Calibration Error (ECE)**: following [3], we partition predictions into B=10 equal-width confidence bins and compute:

$$\text{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left|\text{acc}(b) - \text{conf}(b)\right| \tag{2}$$

  where n_b is the number of samples in bin b, $\text{acc}(b)$ is the bin's accuracy, and $\text{conf}(b)$ is the bin's average confidence.
- **High-Confidence Error Rate (HCE)**: fraction of predictions where the model is *wrong* and has confidence > 0.7. This captures the most dangerous deployment failure: a model that is confidently incorrect.
- **Overconfidence Gap**: average confidence minus accuracy. Values > 0 indicate systematic overconfidence; values < 0 indicate underconfidence; values near 0 indicate good calibration.

4.6. Experimental Design

We evaluate a 2×3 matrix of conditions:

- **Model**: Baseline (zero-shot Llama-3.1-8B-Instruct) *vs*. Fine-tuned (after QLoRA training).
- **Test set**: Test-High *vs*. Test-Low *vs*. Test-Mixed.

This yields 6 experimental conditions, enabling us to isolate both the effect of fine-tuning and the effect of frequency stratification.

5. Experiments and Results

5.1. Accuracy

As shown in Table 2, fine-tuning improves accuracy across all conditions, with the largest gain on the mixed test set (+9.1pp). Overall accuracy rises from 81.9% to 88.7% (+6.8pp).

Hypothesis H1 confirmed: the baseline shows a clear frequency gradient, with high-frequency facts achieving 84.4% accuracy *vs*. 82.3% for low-frequency (a 2.1pp gap). This validates our bucketing approach—even though the baseline never saw FEVER data, it performs better on facts associated with frequently-referenced Wikipedia pages, confirming that FEVER page frequency proxies real-world topic prominence in the LLM's pre-training data.

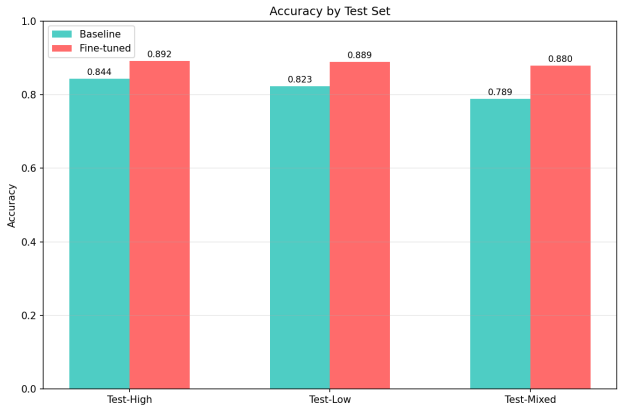

Figure 1. Accuracy comparison. Fine-tuning improves all conditions, with the largest gains on low-frequency and mixed data.

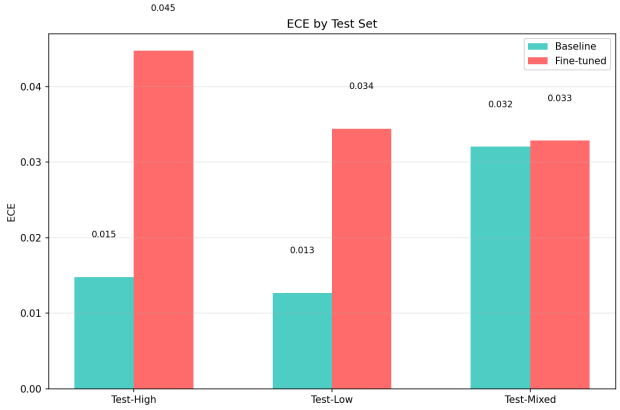

Figure 2. Expected Calibration Error. Baseline is well-calibrated; fine-tuning roughly triples ECE.

	Test-High	Test-Low	Test-Mixed
Average Confidence			
Baseline	0.833	0.815	0.806
Fine-tuned	0.928	0.917	0.912
ECE ↓			
Baseline	0.015	0.013	0.032
Fine-tuned	0.045	0.034	0.033
Overconfidence Gap (closer to 0 is better)			
Baseline	−0.011	−0.008	+0.017
Fine-tuned	+0.036	+0.028	+0.032

Table 3. Calibration metrics. Fine-tuning increases confidence by ∼10pp but accuracy by only ∼7pp, creating systematic overconfidence.

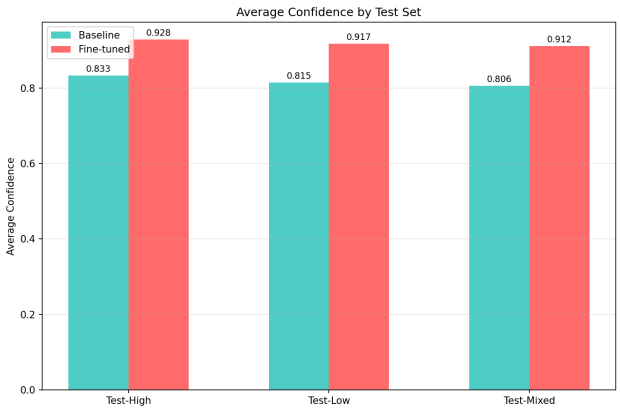

Figure 3. Average confidence. Fine-tuning increases confidence by ∼10pp across all conditions.

Critically, fine-tuning on frequency-balanced data nearly *eliminates* this gap: the fine-tuned model achieves 89.2% on high-frequency *vs*. 88.9% on low-frequency (just 0.3pp difference). The largest accuracy gain occurs on low-frequency facts (+6.6pp *vs*. +4.8pp for high-frequency), suggesting that balanced exposure during fine-tuning specifically helps the model with facts it originally knew less well.

5.2. Confidence and Calibration

Hypothesis H2 confirmed: Table 3 reveals the core accuracy–calibration trade-off. Fine-tuning boosts average confidence by ∼10pp (from ∼0.82 to ∼0.92) but accuracy increases by only ∼7pp, creating a systematic overconfidence gap of ∼3pp.

The baseline model is remarkably well-calibrated: its overall ECE is just 0.013, and its overconfidence gap is essentially zero (−0.001). This means the pre-trained Llama-3.1-8B's confidence scores are almost perfectly reliable out-of-the-box for this task.

Fine-tuning disrupts this calibration. ECE roughly triples on high-frequency data (0.015 → 0.045) and nearly triples on low-frequency data (0.013 → 0.034). The overconfidence gap shifts from slightly negative (marginally underconfident) to consistently positive (+0.028 to +0.036). This aligns directly with the findings of Guo et al. [3]: additional training on modern neural networks pushes confidence up faster than accuracy improves, degrading calibration.

An interesting nuance: the baseline's worst calibration is on the mixed test set (ECE = 0.032), while the fine-tuned model shows relatively *uniform* miscalibration across test sets (ECE 0.033–0.045). Fine-tuning doesn't just degrade calibration—it changes the pattern of miscalibration.

5.3. High-Confidence Errors: The Silver Lining

Hypothesis H3 confirmed: the baseline shows that low-frequency facts produce more high-confidence errors than high-frequency facts (9.1% *vs*. 7.7%), confirming that the

	Test-High	Test-Low	Test-Mixed
Baseline	7.7% (77)	9.1% (91)	9.9% (99)
Fine-tuned	7.2% (72)	6.8% (68)	7.7% (77)
Δ	-0.5pp	-2.3pp	-2.2pp

Table 4. High-confidence error rate (wrong predictions with confidence > 0.7). Absolute counts in parentheses.

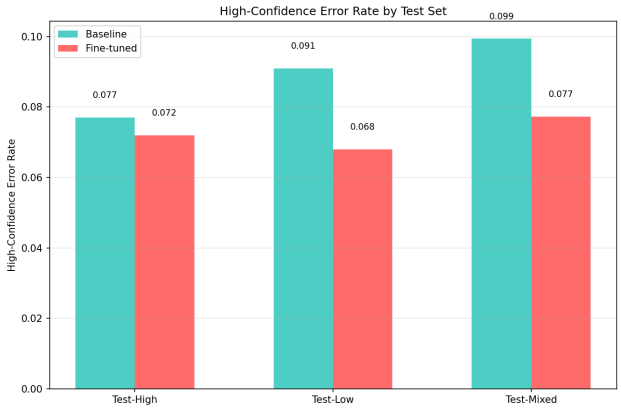

Figure 4. High-confidence error rate. Baseline confirms low-freq vulnerability; fine-tuning fixes it.

model is most dangerously overconfident on facts it has less exposure to.

Despite the overall increase in overconfidence after fine-tuning, the rate of *dangerous* errors actually *decreases* (Table 4). Total HCEs drop from 267 to 217—an 18.7% reduction. The improvement is most pronounced on low-frequency data (-2.3pp, a 25% relative reduction), while high-frequency HCE barely changes (-0.5pp).

This finding reveals a crucial distinction between two types of calibration failure:

- **Global miscalibration**: average confidence $\neq$ average accuracy (captured by ECE). Fine-tuning *worsens* this.
- **Harmful miscalibration**: the model is confidently wrong on specific instances (captured by HCE). Fine-tuning *improves* this.

The fine-tuned model is overconfident on average, but it concentrates its increased confidence on predictions it actually gets right. When wrong, the fine-tuned model's confidence on incorrect predictions averages 0.76 (up from 0.71 in the baseline), but the absolute count of such dangerous errors drops because accuracy improves faster than wrong-prediction confidence rises.

For deployment in safety-critical settings, HCE arguably matters more than ECE: a model that is 3% overconfident on average but rarely makes confident mistakes is preferable to one with perfect ECE but more frequent dangerous errors.

	Supports	Refutes	Gap
Baseline	90.6%	73.2%	17.4pp
Fine-tuned	92.2%	85.2%	7.0pp

Table 5. Per-label accuracy. Fine-tuning cuts the Supports/Refutes gap in half.

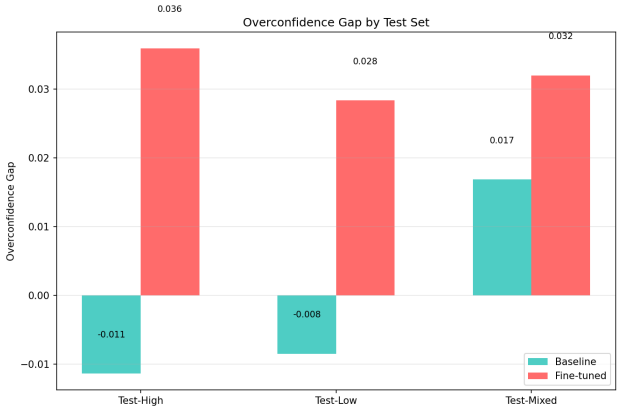

Figure 5. Overconfidence gap by condition. Baseline is well-calibrated (near zero); fine-tuning introduces systematic overconfidence.

5.4. Label Bias Analysis

The baseline exhibits a strong bias toward the Supports class (Table 5), achieving 90.6% accuracy on Supports claims but only 73.2% on Refutes—a 17.4pp gap. This suggests the pre-trained LLM has a "truthfulness prior": when uncertain, it tends to confirm claims rather than refute them.

Fine-tuning dramatically reduces this asymmetry. Refutes accuracy improves by +12.0pp (73.2% $\rightarrow$ 85.2%), while Supports gains only +1.6pp (90.6% $\rightarrow$ 92.2%), cutting the gap to 7.0pp. This debiasing effect is likely due to our balanced training design: equal Supports/Refutes examples within each bucket prevents the model from exploiting label priors.

The remaining 7.0pp gap suggests that some claims flagged as Refutes contain statements that are "almost true" or require nuanced reasoning to reject, making them inherently harder regardless of model bias.

5.5. Calibration Curves

Figure 6 presents reliability diagrams (calibration curves) for all six experimental conditions. These plots partition predictions into confidence bins and compare each bin's average confidence (x-axis) against its actual accuracy (y-axis). Perfect calibration lies on the diagonal.

The baseline model's curves closely track the diagonal across all three test sets, confirming the near-perfect calibration quantified by ECE. The fine-tuned model's curves

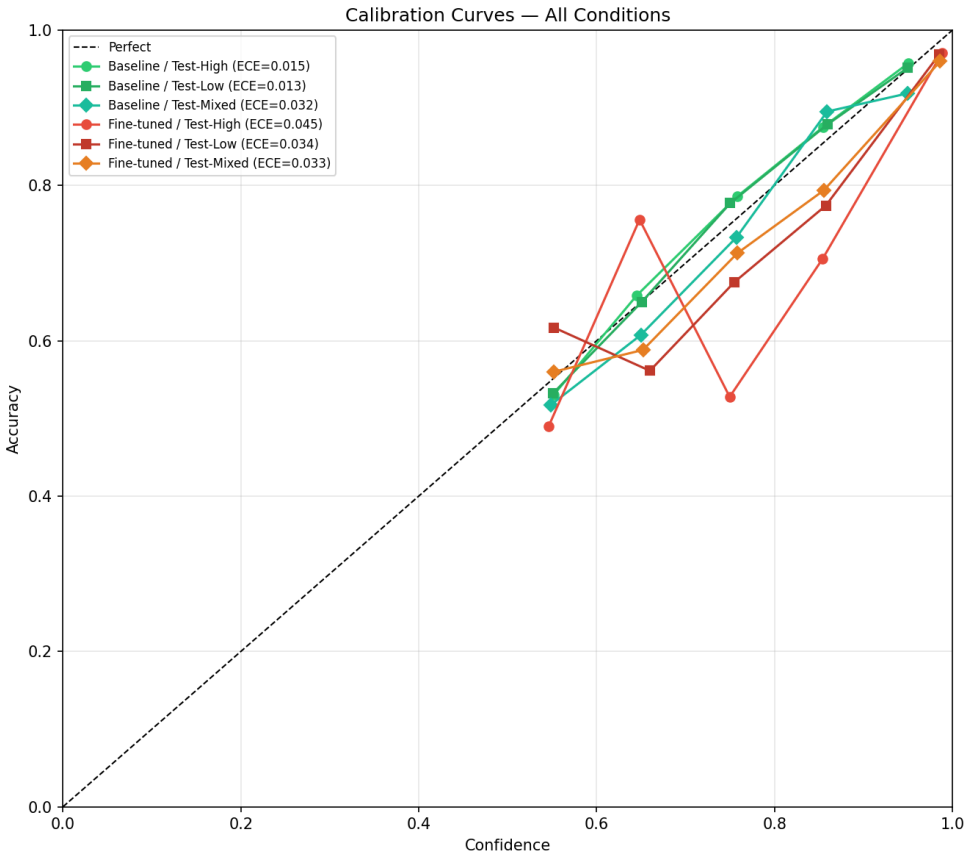

Figure 6. Reliability diagrams for all 6 experimental conditions. Baseline curves (green shades) closely follow the perfect calibration diagonal; fine-tuned curves (red shades) deviate above it in the high-confidence region, indicating systematic overconfidence. The gap between the curves and the diagonal represents the ECE contribution in each bin.

Hypothesis	Result	Evidence
H1: freq $\uparrow \Rightarrow$ acc $\uparrow$	Confirmed	84.4% *vs*. 82.3%
H2: FT $\Rightarrow$ acc $\uparrow$, cal $\downarrow$	Confirmed	+6.8pp acc, 3$\times$ ECE
H3: low-freq $\Rightarrow$ HCE $\uparrow$	Confirmed	9.1% *vs*. 7.7%

Table 6. Summary of hypothesis testing against experimental results.

consistently fall *below* the diagonal in the high-confidence region (> 0.8), indicating that the model assigns confidence scores of $\sim$0.90–0.95 to bins where its actual accuracy is $\sim$0.85–0.90. This visual gap corresponds to the $\sim$3pp overconfidence gap measured quantitatively.

5.6. Summary of Hypothesis Testing

All three hypotheses are confirmed (Table 6). Additionally, we discovered two unexpected findings: (1) fine-tuning reduces HCE despite worsening ECE, and (2) fine-tuning dramatically reduces label bias toward the Supports class.

6. Conclusion

We presented a controlled, data-centric analysis of how training data frequency affects hallucination and confidence calibration in large language models. Using frequency-stratified splits of the FEVER dataset and QLoRA fine-tuning of Llama-3.1-8B-Instruct, our 2$\times$3 experimental design yields five key findings:

1. **Accuracy improves** across all frequency strata (+6.8pp overall), with larger gains on low-frequency facts.
2. **Frequency gap eliminated**: balanced-frequency training reduces the high *vs*. low accuracy gap from 2.1pp to just 0.3pp.
3. **Calibration degrades**: ECE roughly triples and the model becomes systematically overconfident by $\sim$3pp, consistent with [3].
4. **Dangerous errors decrease**: high-confidence error rate drops by 18.7%, with the largest improvement ($-$2.3pp) on low-frequency facts—the most vulnerable category.
5. **Label bias halved**: the Supports/Refutes accuracy gap narrows from 17.4pp to 7.0pp.

The central contribution of our work is identifying the accuracy–calibration trade-off as a function of data frequency, and distinguishing between *global miscalibration* (ECE) and *harmful miscalibration* (HCE). For safety-critical applications, we argue that HCE is the more relevant metric: practitioners should prioritize reducing confidently-wrong predictions over optimizing aggregate calibration statistics.

Limitations. Our study has several limitations: (1) we evaluated a single model architecture (Llama-3.1-8B-Instruct); generalizability to other model families remains to be tested. (2) Wikipedia page frequency is a proxy for pre-training data exposure, not a direct measurement—though the baseline frequency effect validates this proxy. (3) Our binary classification task is simpler than open-ended generation, where hallucination patterns may differ substantially.

Future Work. Several directions merit investigation: (a) applying temperature scaling [3] as a post-hoc calibration correction to recover calibration without sacrificing accuracy; (b) extending to additional model families (Mistral, Gemma) to assess generalizability; (c) testing with more granular frequency buckets or continuous frequency analysis; (d) evaluating on open-ended generation tasks; and (e) exploring curriculum learning strategies that leverage frequency information during training to achieve both accuracy and calibration improvements simultaneously.

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 1, 3

[2] Zorik Gekhman, Gal Yona, Roee Aharoni, Matan Eyal, Amir Feder, Roi Reichart, and Jonathan Herzig. Does fine-tuning LLMs on new knowledge encourage hallucinations? In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. 1, 2

[3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2017. 1, 2, 4, 5, 7, 8

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. 3

[5] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. 1, 2

[6] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. 2

[7] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and VERification. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2018. 1, 2

[8] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817*, 2024. 1, 2

[9] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. Siren's song in the AI ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023. 1, 2

Appendix: Code and Implementation

All code for this project was written in Python using the PyTorch ecosystem. The complete codebase is available at: https://github.com/DCodeGenius/DL-PROJECT

Repository Structure

Key Implementation Details

- **Frequency computation**: We count Wikipedia page references across the *entire* FEVER training set (not just our sampled subset), then compute percentile thresholds on the *unique page* frequency distribution to avoid skew from pages like "United States" that generate hundreds of claims.
- **Balanced sampling**: Within each frequency bucket, we use stratified random sampling to draw equal numbers of Supports and Refutes examples, preventing the model from learning to exploit label priors.
- **Confidence extraction**: Rather than using generation-based confidence (which requires sampling and can be noisy), we extract logits at the final position for the two specific label tokens and apply softmax, providing a deterministic probability estimate.
- **No evidence provided**: The prompt deliberately withholds Wikipedia evidence, forcing the model to rely entirely on parametric knowledge. This isolates the effect of pre-training exposure frequency on factual recall accuracy and confidence.
- **Reproducibility**: All random operations use seed 42. Test sets use offset seeds (43, 44, 45) to ensure independence from training sampling.

File	Description
`config.py`	Central configuration: model ID (`meta-llama/Meta-Llama-3.1-8B-Instruct`), LoRA parameters (r=16, α=32, dropout 0.05), training hyperparameters (2 epochs, LR 2×10^{-4}, cosine scheduler), NF4 quantization settings, frequency bucketing percentile thresholds (33/67).
`preprocess_data.py`	Complete data pipeline: loads FEVER v1.0 from HuggingFace, filters to binary classification, computes Wikipedia page reference frequencies across the full training set, derives percentile-based bucketing thresholds on unique page frequencies, constructs balanced train/test splits with stratified sampling, and builds claim-only instruction prompts.
`train.py`	Fine-tuning script: loads Llama-3.1-8B-Instruct with 4-bit NF4 quantization via BitsAndBytes, prepares model for k-bit training, attaches LoRA adapters to all 7 target modules (q/k/v/o_proj + gate/up/down_proj), trains with HuggingFace Trainer using causal LM objective and cosine LR schedule.
`evaluate.py`	Evaluation script: loads either baseline (zero-shot) or fine-tuned model with same quantization config, iterates over test set metadata, extracts SUPPORTS/REFUTES logits at final token position, computes softmax confidence, saves per-example results with claims, predictions, confidence scores, and metadata.
`analyze.py`	Analysis and visualization: loads all 6 evaluation result files, computes accuracy/ECE/HCE/overconfidence gap per condition, generates 5 grouped bar charts (2×3), 6 individual reliability diagrams, and 1 combined calibration overlay, outputs comprehensive JSON analysis file.
`run_all_evals.py`	Efficient batch evaluation: loads each model (baseline/fine-tuned) once and evaluates on all 3 test sets before unloading, reducing GPU memory churn compared to running `evaluate.py` 6 times separately. Runs analysis automatically after all evaluations.
`run_full_pipeline.sh`	End-to-end shell automation: preprocess → baseline eval (3 sets) → fine-tune → fine-tuned eval (3 sets) → analysis → auto-stop GCP VM to minimize cloud costs.
`requirements.txt`	Python dependencies: torch ≥2.0, transformers 4.44.2, peft 0.13.2, bitsandbytes 0.41.1, datasets, accelerate 0.34.2, matplotlib, scikit-learn, tqdm.

Project file descriptions. All experiments ran on a Google Cloud Platform VM with NVIDIA L4 GPU (24GB VRAM). Full pipeline runtime: ∼3.5 hours.